📘 CORE JAVA FIRST-ROUND INTERVIEW HANDBOOK

(Questions with crisp answers)

1️⃣ INTRO & RESUME QUESTIONS

Q1. Introduce yourself.

Answer:
I'm Rohit Jangir, a Software Engineer with 3+ years of experience in Java and Spring Boot. I work on scalable microservices, async processing using RabbitMQ, and cloud deployments with Docker and Kubernetes on AWS. I've handled performance optimization, Java 17 migration, and multi-tenant architecture. I enjoy working on system performance, concurrency, and clean backend design.

Q2. What do you do in your current role?

Answer:
I design and develop backend microservices using Java and Spring Boot, handle async communication with RabbitMQ, optimize database performance, and manage containerized deployments on AWS using Docker and Kubernetes.

2️⃣ CORE JAVA – MUST-ASK QUESTIONS

Q3. How does HashMap work internally?

Answer:
HashMap uses the **hashCode()** of the key to calculate a bucket index.
Each bucket stores entries as a **linked list**, which becomes a **red-black tree** if collisions exceed a threshold.
When retrieving, HashMap uses **hashCode → bucket → equals()** to find the exact key.

Q4. Why HashMap is not thread-safe?

Answer:
Because multiple threads can modify it simultaneously, causing data inconsistency, infinite loops during resize, or lost updates.

Q5. Difference between HashMap and ConcurrentHashMap?

Answer:

HashMap	ConcurrentHashMap
Not thread-safe	Thread-safe
Entire map can break	Bucket-level locking
Faster in single thread	Safe for concurrency

Q6. What is synchronized?

Answer:
synchronized ensures **only one thread** can access a critical section at a time by acquiring a monitor lock.

Q7. volatile vs synchronized?

Answer:

volatile	synchronized
Visibility only	Visibility + atomicity
No locking	Uses locking
Faster	Slower

Q8. Why String is immutable?

Answer:
For **security**, **thread safety**, and **string pool optimization**.
Immutability prevents accidental modification and enables reuse.

Q9. String vs StringBuilder vs StringBuffer?

Answer:

- String → Immutable
- StringBuilder → Mutable, not thread-safe
- StringBuffer → Mutable, thread-safe

Q10. ArrayList vs LinkedList?

Answer:

ArrayList	LinkedList
Fast random access	Slow random access
Slow insert/delete	Fast insert/delete
Less memory	More memory

Q11. equals() and hashCode() contract?

Answer:
If two objects are equal using equals(), they **must have the same hashCode**.

3️⃣ MULTITHREADING (SCREENING LEVEL)

Q12. Thread vs Runnable?

Answer:
Runnable separates **task from execution**, supports multiple inheritance, and is preferred for scalability.

Q13. What is a race condition?

Answer:
When multiple threads access shared data simultaneously, causing inconsistent results.

Q14. What is deadlock?

Answer:
When two or more threads wait indefinitely for each other's resources.

Q15. How do you prevent deadlock?

Answer:

- Avoid nested locks
- Maintain lock ordering
- Use timeout-based locks

Q16. wait() vs sleep()?

Answer:

- wait() releases the lock
- sleep() does not release the lock

Q17. What is ExecutorService?

Answer:
A framework that manages thread pools efficiently instead of creating threads manually.

🧠 JAVA MEMORY & JVM

Q18. Stack vs Heap?

Answer:

- Stack → Method calls, local variables (thread-safe)

- Heap → Objects (shared across threads)

Q19. What is Garbage Collection?

Answer:
Automatic process of reclaiming unused objects from heap memory.

Q20. What causes memory leaks?

Answer:

- Static references
- Listeners not removed
- Unbounded caches
- ThreadLocal misuse

5️⃣ JAVA 8+ (VERY COMMON)

Q21. What is Lambda?

Answer:
An anonymous function used to implement functional interfaces concisely.

Q22. What is a Functional Interface?

Answer:
An interface with **exactly one abstract method**.

Q23. Why Streams are lazy?

Answer:
Stream operations execute only when a terminal operation is called, improving performance.

Q24. Optional vs null?

Answer:
Optional prevents NullPointerException and forces explicit handling.

6️⃣ CODING (SIMPLE LOGIC)

Q25. Reverse a string.

Answer (Logic):

- Convert to char array
- Swap characters from start and end
- Stop at middle

Q26. Find duplicate in an array.

Answer:
Use HashSet or HashMap to track frequency.

Q27. Count character frequency.

Answer:
Iterate string → use HashMap<Character, Integer>

7️⃣ BEHAVIORAL (SHORT ANSWERS)

Q28. Toughest bug you fixed?

Answer:
A concurrency issue causing duplicate processing. Fixed using idempotency and synchronization at consumer level.

Q29. Performance issue you handled?

Answer:
Optimized database queries and async processing, improving system performance by ~20%.

Q30. Why are you looking for a change?

Answer:
I want to work on problem-heavy systems where performance, concurrency, and system design are core to the product.

🛂 QUESTIONS YOU SHOULD ASK THEM

Ask **one**:

- "What kind of problems will I work on in the first 6 months?"
- "How does the team handle performance and reliability challenges?"

✅ FINAL FIRST-ROUND RULE

If you:

- Answer **HashMap + threading + one coding question**
- Speak clearly
- Show real experience

👉 **You WILL clear round 1.**

If you want next:

- 🔹 **Mock 30-min interview (exact format)**
- 🔹 **Ultra-short 1-page revision**
- 🔹 **Second-round prep**

Just say the word 👌

Give coding examples as well with code

Perfect 👍
Below is a **CODING HANDBOOK (Core Java – First Round)** with **actual Java code**, exactly at the **difficulty level asked in a 30-min screening round**.

👉 These are **whiteboard / screen-share friendly** and **interviewer-approved**.

📘 CORE JAVA CODING HANDBOOK

(Questions + Clean Java Code + Explanation)

1️⃣ Reverse a String

Question

Reverse a string without using extra libraries.

Code

```
public class ReverseString {

   public static void main(String[] args) {

      String str = "Rohit";

      char[] arr = str.toCharArray();


      int left = 0, right = arr.length - 1;

      while (left < right) {

         char temp = arr[left];

         arr[left] = arr[right];

         arr[right] = temp;

         left++;

         right--;
```

```
    }

    System.out.println(new String(arr));
  }
}
```

Key Points to Say

- Time Complexity: **O(n)**
- Space Complexity: **O(n)** (char array)
- In-place swap logic

2️⃣ Find Duplicate Elements in an Array

Question

Find duplicate numbers in an array.

Code (Using HashSet)

```
import java.util.HashSet;
import java.util.Set;

public class FindDuplicates {
  public static void main(String[] args) {
    int[] arr = {1, 2, 3, 4, 2, 3};

    Set<Integer> set = new HashSet<>();
    for (int num : arr) {
      if (!set.add(num)) {
        System.out.println("Duplicate: " + num);
      }
```

```
    }
  }
}
```

Key Points

- HashSet doesn't allow duplicates
- add() returns false if element exists
- Time: **O(n)**

3️⃣ Count Character Frequency in a String

Question

Count frequency of each character.

Code

```
import java.util.HashMap;
import java.util.Map;

public class CharFrequency {
  public static void main(String[] args) {
    String str = "interview";

    Map<Character, Integer> map = new HashMap<>();
    for (char c : str.toCharArray()) {
      map.put(c, map.getOrDefault(c, 0) + 1);
    }

    System.out.println(map);
  }
```

}

Key Points

- getOrDefault() avoids null checks
- Time: **O(n)**
- Space: **O(k)** (unique chars)

4️⃣ Find First Non-Repeating Character

Question

Return first character that doesn't repeat.

Code

```
import java.util.LinkedHashMap;
import java.util.Map;

public class FirstNonRepeating {
  public static void main(String[] args) {
    String str = "swiss";

    Map<Character, Integer> map = new LinkedHashMap<>();
    for (char c : str.toCharArray()) {
      map.put(c, map.getOrDefault(c, 0) + 1);
    }

    for (Map.Entry<Character, Integer> entry : map.entrySet()) {
      if (entry.getValue() == 1) {
        System.out.println(entry.getKey());
        break;
```

```
        }
    }
  }
}
```

Why LinkedHashMap?

- Maintains insertion order

5️⃣ Check if Two Strings Are Anagrams

Question

Check if two strings are anagrams.

Code

```
import java.util.Arrays;


public class AnagramCheck {
  public static boolean isAnagram(String a, String b) {
    if (a.length() != b.length()) return false;

    char[] arr1 = a.toCharArray();
    char[] arr2 = b.toCharArray();

    Arrays.sort(arr1);
    Arrays.sort(arr2);

    return Arrays.equals(arr1, arr2);
  }
```

```
    public static void main(String[] args) {
        System.out.println(isAnagram("listen", "silent"));
    }
}
```

6️⃣ Simple Thread-Safe Counter (Concurrency Favorite)

Question

Create a thread-safe counter.

Code (Using synchronized)

```
public class Counter {
    private int count = 0;

    public synchronized void increment() {
        count++;
    }

    public int getCount() {
        return count;
    }
}
```

What to Say

- synchronized ensures atomicity
- Prevents race condition

7️⃣ Producer–Consumer (Basic Version)

Question

Explain Producer–Consumer using wait/notify.

Code

```
class SharedResource {
   private int data;
   private boolean available = false;

   public synchronized void produce(int value) throws InterruptedException {
      while (available) {
         wait();
      }
      data = value;
      available = true;
      notify();
   }

   public synchronized int consume() throws InterruptedException {
      while (!available) {
         wait();
      }
      available = false;
      notify();
      return data;
   }
}
```

Key Concepts

- wait() releases lock

- notify() wakes waiting thread
- Avoids busy waiting

8️⃣ ExecutorService Example

Question

Why use ExecutorService?

Code

```
import java.util.concurrent.ExecutorService;

import java.util.concurrent.Executors;


public class ExecutorExample {

  public static void main(String[] args) {

    ExecutorService executor = Executors.newFixedThreadPool(2);


    executor.submit(() -> System.out.println("Task 1"));

    executor.submit(() -> System.out.println("Task 2"));


    executor.shutdown();

  }

}
```

Key Points

- Reuses threads
- Better performance than new Thread()

9️⃣ Remove Duplicates from List

Code

```
import java.util.*;


public class RemoveDuplicates {
   public static void main(String[] args) {
      List<Integer> list = Arrays.asList(1, 2, 2, 3, 4, 4);


      Set<Integer> set = new LinkedHashSet<>(list);
      System.out.println(set);
   }
}
```

🔟 LRU Cache (Logic-Level – Interview Gold)

Question

Explain LRU Cache (no full code required in round 1).

Core Idea

- HashMap → O(1) access
- Doubly Linked List → track usage order

Key Line to Say

“Most recently used goes to the tail, least recently used is removed from the head.”

✅ FINAL SCREENING ROUND CODING STRATEGY

If interviewer asks to code:

1. Clarify input/output
2. Write clean logic
3. Explain time & space complexity
4. Handle edge cases

5. Don’t rush